

W++

A programming language with its own compiler frontend, targeting Python.
Technical documentation.

version 1.1.0 | 19 keywords | 252 tests
github.com/harrynoble/wplusplus
wplusplus.vercel.app

01 What W++ is

W++ is a small programming language whose nineteen keywords are Gen Z slang: cook declares a function, yap prints, bet is if. It is dynamically typed and built on Python semantics.

What makes it a language implementation rather than a text substitution is the frontend. W++ source is tokenized, parsed into a W++ syntax tree, checked against W++'s own rules, and only then translated into Python by a separate backend. Python is the execution target. W++ has no runtime of its own and does not compile to machine code.

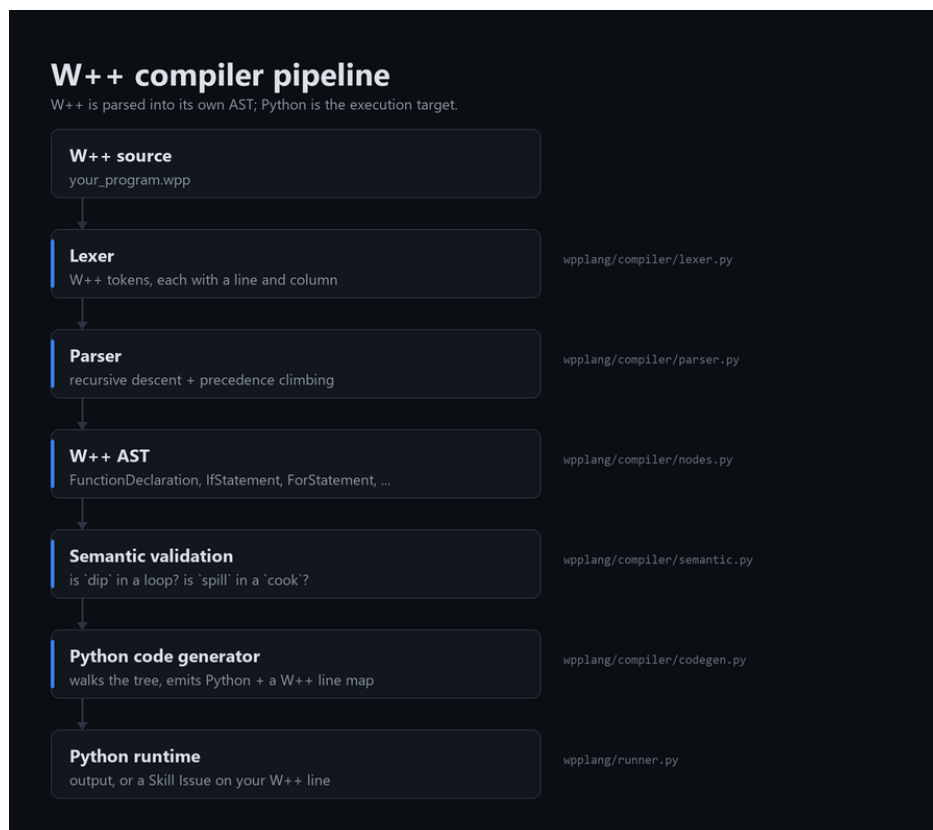
It runs two ways from one codebase: from the command line, and in a browser playground that can be deployed as static files because the compiler itself runs client-side under Pyodide.

At a glance

Compiler frontend	2654 lines	lexer, parser, AST, semantic pass, code generator
Language and CLI	667 lines	keyword table, runner, error protocol, entry point
Tests	252 tests	3414 lines, standard library unittest
Dependencies	none	for the language; Pyodide and Monaco in the browser only
Examples	6 programs	including the two from the language specification

02 Architecture

Each stage has one responsibility and hands a single kind of thing to the next. No stage rewrites W++ text into Python text; the two vocabularies meet in exactly one place, the keyword table, which the code generator consults as it walks the tree.



The stages marked in blue are the compiler proper. Everything else - the CLI, the playground, the execution layer - was already there and did not change when the compiler replaced the original text translator.

03 Inside the compiler

Lexer

Produces W++ tokens carrying a line and a column. W++ and Python are lexically identical - same numbers, strings, operators and indentation rules - and differ only in which bare words are keywords, which is a parser question. So character scanning is delegated to Python's keyword-agnostic `tokenize`, and this stage adds what is W++'s own: classifying the nineteen keywords, collapsing f-strings so their replacement fields can be parsed separately from their text, and reporting a scanning failure as a W++ error.

One consequence is structural rather than cosmetic. A keyword inside a string can never be translated, because the scanner returns the whole literal as one token - nothing downstream ever looks inside it.

Parser and AST

Recursive descent for statements, one method per construct; precedence climbing for expressions, so the precedence table is written once. The tree describes W++ constructs and records which W++ keyword opened each one. Nothing in it holds half-translated text.

```
cook check_vibe(name):
    bet name == "Claude":
        spill "W AI"
    nah:
        spill "Mid"
```

```
Program @1:0
  body:
    FunctionDeclaration name='check_vibe' keyword='cook' @1:0
      params:
        Parameter name='name' kind='normal' @1:16
      body:
        IfStatement @2:4
          branches:
            (
              ComparisonExpression operators=['=='] @2:8
                left:
                  Identifier name='name' @2:8
                comparators:
                  Literal raw='"Claude"' kind='string' @2:16
              ReturnStatement keyword='spill' @3:8
            )
          or_else:
            ReturnStatement keyword='spill' @5:8
```

The same program before and after parsing. Printed by python `wpp.py --ast`; the tree for the two lines after the function continues past what is shown.

Semantic pass

The questions that need a view of the whole tree: is `dip` inside a loop, is `spill` inside a `cook`, is a parameter repeated, is a keyword being used where only a name can go. Deliberately small - W++ is dynamically typed, so there is no type checking to do and inventing some would change the language.

Code generator

Walks the tree and emits Python, lining each statement up with the W++ line it came from and recording the mapping. That map is what lets a Python exception be reported against the author's source.

04 Errors: the Skill Issue Protocol

Python exceptions are intercepted and re-presented in W++'s own wording, with the W++ line that caused them, the source line itself, and Python's original description underneath.

```
Math ain't mathing: Bro tried to divide by zero (ZeroDivisionError)
  where: average.wpp, line 5
        5 | spill total / bodycount(marks)
  details: division by zero
```

Python exception	W++ message
SyntaxError	Negative Aura: Bro forgot how to type (SyntaxError)
NameError	Bro is making up words now (NameError)
TypeError	Oil up bro, you can't combine those (TypeError)
IndexError	Blud thinks he has more items than he does (IndexError)
ZeroDivisionError	Math ain't mathing: Bro tried to divide by zero (ZeroDivisionError)
IndentationError	Your spaces are looking a little sus (IndentationError)
KeyboardInterrupt	Go touch grass, you've been looping forever (KeyboardInterrupt)

Anything outside this table is reported as Unspecified skill issue with the real exception name, so a raw traceback never reaches the user. Errors the frontend finds - a keyword used as a name, `break` outside a loop - travel the same path, and name the offending word.

Reporting the author's line is the practical reason the AST exists. A runtime failure knows only the generated Python line; the source map turns it back into the W++ line, so the message never refers to code nobody wrote.

05 The playground, and how it deploys

One interface, two execution engines. Which one is used is decided at load time by asking whether a backend answers, and the interface cannot tell them apart because both emit the same records.

Engine	Where W++ runs	Used when
Server	playground/server.py, one child process per program	running locally
Browser	the same wpplang package under Pyodide, in a Worker	no backend answers

Because the browser engine needs no backend, the site is static files with no build step: the compiler's Python sources are committed as a JSON bundle and fetched at load. That is also what makes it safe to host publicly - there is no server executing anyone's code. The local server, which does execute code with only a timeout and no sandbox, binds to 127.0.0.1 deliberately and is not what gets deployed.

Behaviour shared by both engines

- Programs stop after ten seconds of running. Time spent waiting for input does not count, so answering slowly is fine while a runaway loop is still stopped.
- A program that calls `dm()` prints its prompt into the output panel and takes the answer typed straight after it, as a terminal would.
- Errors appear with the W++ line, the source line, and the failing line marked in the editor.

The browser engine differs in two ways, both deliberate. Interactive input works by replay - a worker cannot pause synchronous Python for the user without cross-origin isolation headers, so the program is run again from the start with each new answer and the transcript rebuilt; each attempt takes milliseconds. And a runaway loop is stopped by discarding the worker, which is the only way to interrupt synchronous Python in a browser.

06 How we know it works

252 tests, standard library only.

Suite	Covers
<code>test_compiler.py</code>	each stage on its own: lexer, parser and AST shape, semantic rules, code generation, source mapping
<code>test_compiler_equivalence.py</code>	the refactor itself - see below
<code>test_programs.py</code>	whole programs with exact expected output, including quicksort, a sieve, BFS, and diagnostics for errors raised inside comprehensions, generators and decorated functions
<code>test_translator.py</code>	keyword mapping, word boundaries, literal safety, line numbering
<code>test_errors.py</code>	every Skill Issue message and its context
<code>test_playground.py</code>	the API, interactive prompts, the compute budget, Stop, runaway output, worker cleanup
<code>test_web_bundle.py</code>	the browser bundle still matches the package, including by importing the compiler from the bundled text alone
<code>test_cli.py</code> , <code>test_sound.py</code> , <code>test_guide.py</code>	the command line, the error sound, and every example in the learning guide

The equivalence check

W++ was originally implemented as a single regular-expression pass over the source. When that was replaced by the compiler, the old translator was kept as a test oracle - on no execution path, with a test asserting nothing in the package imports it.

Both target Python, so for any program the two both accept, they must generate Python that parses to the same tree. Comparing parsed trees rather than text ignores layout and redundant brackets, which are the generator's business, and catches anything that would change behaviour. The check runs on every shipped example and on a program for every construct the language supports, and they agree on all of them. The only disagreement anywhere is a program the compiler now deliberately rejects - see section 07.

That check is what made the refactor safe to do at all, and it found the two real bugs in it: a nah attached to a loop being claimed by an inner bet - which silently changed what programs meant - and nested f-strings failing to lex.

07 Decisions worth recording

Keywords are reserved words

You cannot name a function, class, parameter or keyword argument after one, for the same reason Python will not let you write `def break(): cook dip(self)` would become `def break(self)`, and a call written `dip()` would become `break()` anyway. The compiler says which word is the problem and on which line, rather than letting Python complain about generated code. Names that merely contain a keyword are fine, and so are attributes - a name after a dot is never a keyword.

Two changes from the original implementation

Both narrow invalid programs only, and both are documented:

- The parser needs whole statements. `cook squad(x):` with no body is refused; the regular-expression version accepted it because it never parsed anything.
- A keyword used where a name is required is refused up front. `f(bet=1)` previously became `f(if=1)` and failed confusingly.

Kept, not rewritten

The CLI, the execution layer, the playground and its API contract were already working and were left alone. The compiler was slotted in behind the same `translate()` entry point, which is why nothing above it had to change.

08 What it does not do

- Only the nineteen keywords have slang names. `and`, `class`, `import`, `try` and the rest are written the Python way.
- One file per program. Python modules can be imported; another `.wpp` file cannot.
- No REPL, and errors stop at the first failure, as in Python.
- Keywords are source syntax, not a display layer: `yap(nocap)` prints `True`.
- `case` patterns are carried through as written, since `W++` adds nothing to Python's pattern syntax and a name there means `bind` rather than `read`.
- The browser engine downloads Pyodide on first load - tens of megabytes, a few seconds - and its replay approach to input can differ for a program whose output depends on randomness or the clock.
- The local server executes the code it is given with only a timeout. It is a development tool, not a sandbox.

How it was built

In order, from the repository history:

Commit		Commit	
1	feat: implement W++ Python translator	8	Update README
2	feat: add W++ web playground	9	feat: add a favicon for the playground
3	test: add a keyword tour exercising all 19 keywords	10	feat: introduce AST-based W++ compiler
4	fix: interactive input in the output panel	11	feat: run W++ in the browser so the playground can deploy statically
5	fix: runaway loops, and correctness fixes found by hard programs	12	docs: rewrite the README in the TinkerHub submission format
6	docs: add the complete W++ learning guide as a PDF	13	docs: add real screenshots, the deployment link and AI credits
7	feat: reword the ZeroDivisionError skill issue, and add an error sound	14	fix: the error sound was missing from the deployed playground

The language and its tests came first; the playground, the guide and the compiler refactor followed. The refactor is the only change that touched the core, and the equivalence check in section 06 is why it could be done without risking the working demo.